TradeIO

D0 High-Level System Design

Senior Design

Team Members

Ishaa Jain

Samarth Prajapati

Project Goal

TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

1. Title, Goal Statement, and Conventions

Project Title

TradeIO — Trading Journal and Behavioral Analytics Platform

Goal Statement

TradeIO is a trade journal for traders that provides the ability to record trading actions, evaluate execution and psychology, track risks and discipline, and gain valuable insights about trading.

This software is aimed at helping users not only analyze their trading outcomes but also to comprehend what behavior led to those outcomes.

Basic Input

TradeIO inputs include the following: user-entered trading data, reflections on journals, risk settings, objectives, psychological data, and account preferences.

Basic Output

The fundamental outputs include trading history database, performance metrics, risk and behavior alerts, progress toward goals data, dashboards, and optionally, AI coaching insights.

Diagram Conventions

The diagrams in this document use the following conventions:

- **Solid white boxes** denote those components that will be designed and implemented by the TradeIO team.
- **Gray dashed boxes** denote external systems or third-party components.
- **Arrows** denote the flow of data or request/reply from one component to another.
- Interface identifiers such as **I1**, **I2**, and **I3** correspond directly to the interfaces defined in Section 4.
- Labels beside arrows identify the interface or data being transferred.

2. Block Diagram (D0 Diagram)

TradeIO — D0 High-Level System Design

Goal: Allow traders to track their trades, evaluate their execution and psychology, control their risks and discipline, and gain important insights into performance and behavior.

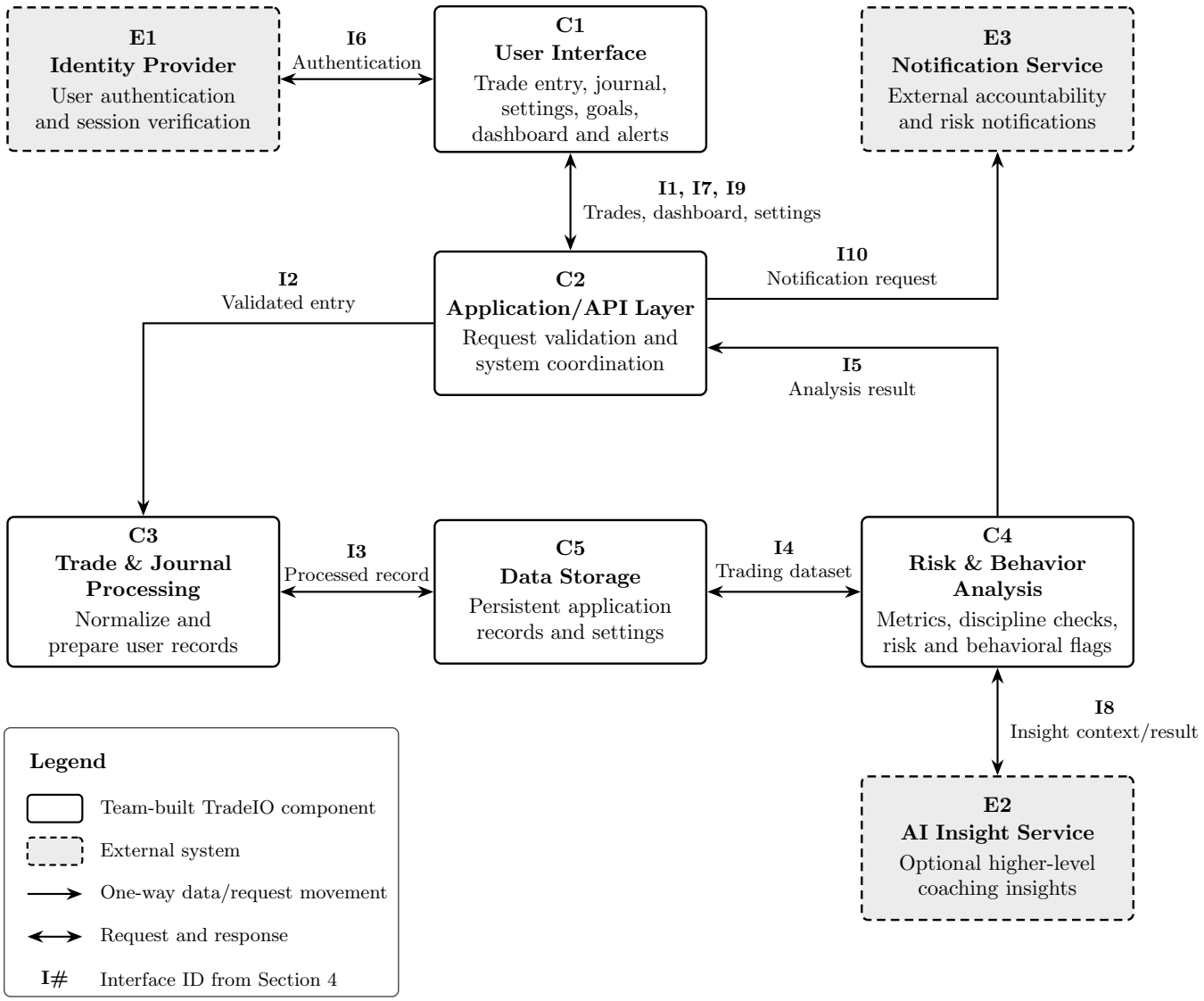


Figure 1: TradeIO D0 High-Level Block Diagram

External systems are shown with dashed gray borders; all solid white boxes are TradeIO team-built components.

3. Component Responsibility Table

ID	Component	Responsibility	Interfaces In	Interfaces Out	Primary Owner
C1	User Interface	Collects user input and presents trade, journal, risk, goal, dashboard, and insight information.	I6, I7	I1, I9	Samarth
C2	Application/API Layer	Validates requests and coordinates communication between the user interface and internal system components.	I1, I5, I9	I2, I7, I10	Ishaa
C3	Trade & Journal Processing	Transforms validated trade, journal, goal, and reflection input into normalized records suitable for storage.	I2	I3	Ishaa
C4	Risk & Behavior Analysis	Evaluates stored trading information against risk, discipline, psychology, and behavioral rules to produce metrics and alerts.	I4, I8	I5, I8	Samarth
C5	Data Storage	Persists user settings, trade records, journal entries, goals, and derived analysis information.	I3	I4	Ishaa
E1	Identity Provider	Authenticates users and provides verified identity and session information.	I6	I6	External
E2	AI Insight Service	Produces optional coaching text from prepared behavioral and performance summaries.	I8	I8	External
E3	Notification Service	Delivers configured accountability or risk notifications outside the TradeIO application.	I10	I10	External

4. Interface Specification Table

ID	Connection	Inputs / Outputs	Format	Protocol	Error Handling
I1	UI → Application/API	Input: TradeEntryRequest {instrument:string, side:enum, entryPrice:number, exitPrice?:number, quantity:number, leverage?:number, notes?:string}. Output: TradeEntryResponse {tradeId:string, status:enum}.	JSON	HTTPS request/response	If C2 receives an incorrect request or an invalid request, then it will generate an error.
I2	Application/API → Trade & Journal Processing	Input: ValidatedEntry {userId:string, entryType:enum, fields:object}. Output: ProcessedRecord {recordId:string, normalizedFields:object, warnings:string[]}.	JSON / internal object	Internal service call	C3 returns a structured validation failure. Records that are not valid are not stored.
I3	Trade & Journal Processing ↔ Data Storage	Input: ProcessedRecord {recordId:string, userId:string, type:enum, data:object}. Output: StoredRecord {id:string, createdAt:datetime}.	Structured record	Database transaction	C5 rolls back the writes that fail and gives a storage error. The application does not report the record as successful.
I4	Data Storage ↔ Risk & Behavior Analysis	Input: TradingDataset {trades:TradeRecord[], journal:JournalRecord[], settings:RiskSettings}. Output: AnalysisRecord {period:string, metrics:object, flags:string[]}.	Structured records	Database query	Records that are incomplete can be overlooked where appropriate. Failure of queries will not allow the publication of a new result of analysis.
I5	Risk & Behavior Analysis → Application/API	Output: AnalysisResponse {metrics:object, alerts:Alert[], behaviorSummary:object}.	JSON	Internal service call	C4 gives an analysis result status. While C2 denotes that the analysis is not possible, and does not generate unsupported results.
I6	TradeIO ↔ Identity Provider	Input: AuthRequest {sessionToken:string}. Output: AuthResult {userId:string, authenticated:boolean}.	Token / JSON claims	HTTPS / authentication	Invalid or expired sessions are rejected. Protected requests are blocked until the user authenticates again.
I7	Application/API → User Interface	Input: DashboardRequest {period:string}. Output: DashboardResponse {trades:object[], metrics:object, goals:object[], alerts:object[]}.	JSON	HTTPS request/response	C2 returns an error status and the UI displays a retry state instead of displaying incomplete data as current.
I8	Risk & Behavior Analysis ↔ AI Insight Service	Input: InsightContext {period:string, performanceSummary:object, behaviorSummary:object}. Output: InsightResult {summary:string, suggestions:string[]}.	JSON	HTTPS API	AI output is optional. A timeout or external service failure does not prevent core analytics from being displayed.
I9	User Interface → Application/API	Input: SettingsRequest {timezone:string, currency:string, riskLimits:object, notificationPrefs:object}. Output: SettingsResponse {saved:boolean, effectiveAt:datetime}.	JSON	HTTPS request/response	C2 validates allowed values and returns field-level errors. Invalid settings are not saved.
I10	Application/API → Notification Service	Input: NotificationRequest {userId:string, channel:enum, eventType:enum, message:string}. Output: DeliveryResult {status:enum, providerId?:string}.	JSON	HTTPS API	Delivery failures are recorded while the in-app warning remains available. Notification failure does not change trading records.

Example Interface Payload

The following example represents an **I1 TradeEntryRequest**:

```
{
  "instrument": "XAUUSD",
  "side": "LONG",
  "entryPrice": 2650.50,
  "exitPrice": 2661.00,
  "quantity": 0.05,
  "leverage": 10,
  "notes": "Followed planned entry and stop."
}
```

5. Data-Flow Diagram

TradeIO — Data-Flow Diagram

Goal: Transform user trading and reflection data into validated and stored records, behavioral and risk analysis, and useful feedback for the trader.

Flow A — Trade and Journal Processing

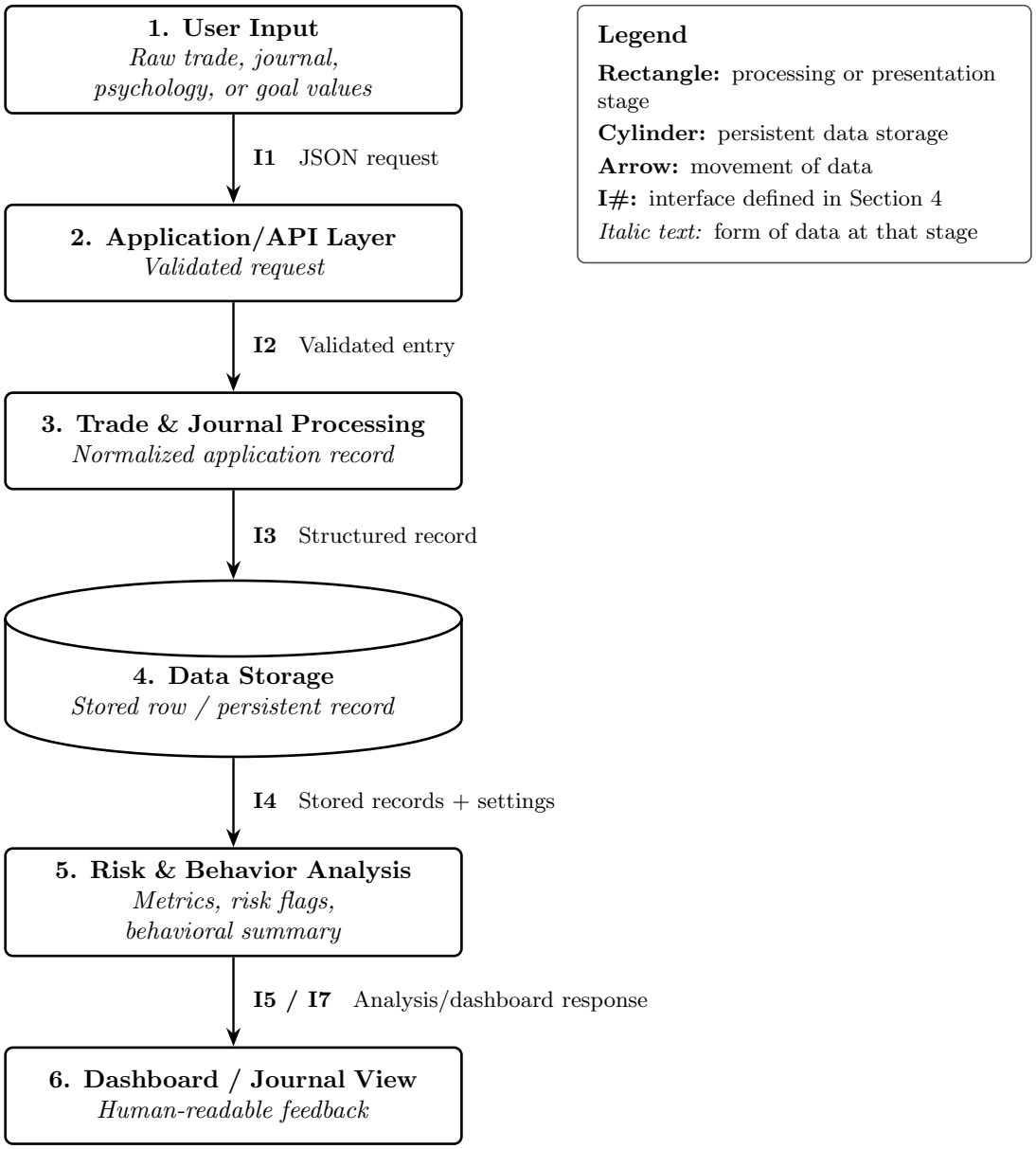


Figure 2: Primary Trade and Journal Data Flow

Flow B — Optional AI Insight and Accountability

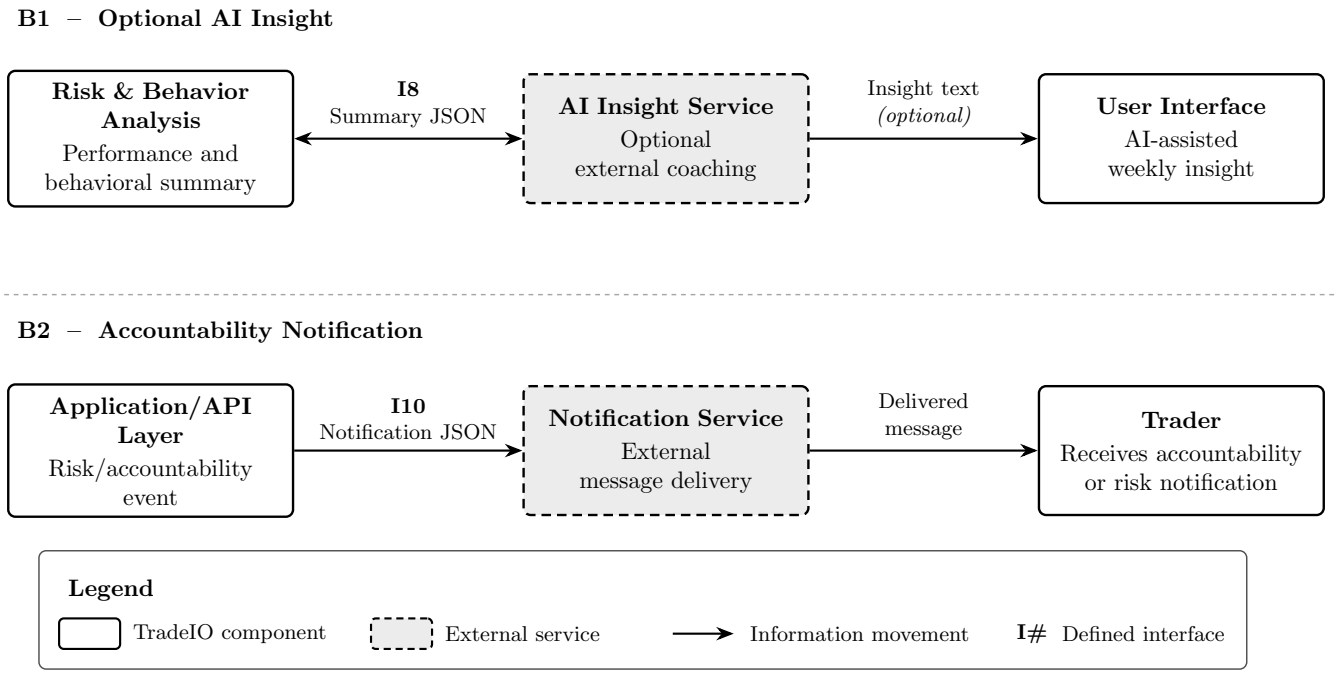


Figure 3: Optional AI Insight and Accountability Data Flow

Timing Budget

There is no strict real-time deadline in the present high-level requirements.

Core functions like trade logging, journal creation, record retrieval, and dashboard viewing must operate within standard web application response times.

External artificial intelligence-based insight creation and notification delivery are considered supplementary functions. Any failure or delay in the external process will not stop the storage of trade or journal record or stop the presentation of deterministic analytics.

6. Architecture Pattern and Justification

TradeIO basically uses the **Client-Server Architecture** together with a **Layered Architecture** for the server-side part of the system.

Also, there is a conceptual use of the **Pipeline Pattern**, which transforms user trading into normalized data and later to behavior/risk analysis.

User Interface is the client. The Application/API Layer manages all server-side functions. Trade processing, behavior analysis, and data storage are kept separate according to their roles.

Fit to the Problem

TradeIO is an interactive application wherein the user can create, alter, analyze, and access persistent trading data in multiple sessions.

A client server system provides data access, authentication, and consistent operation of the application.

The layering design keeps the presentation, request, trades, analyses, and storage separate. Thus, any part of the system may evolve independently.

Team Skills

This project is developed by a Computer Science team of two people.

Client/Server approach and layers provide a structure that applies principles of software development known to the team but at the same time provides separation between the main components of the application.

Component boundaries also offer obvious areas of responsibility which will be used for implementation tasks in the future.

Performance and Timing

Trade logging, generation of journal entries, and deterministic risk analysis are all contained within the main TradeIO system.

The optional external functionality, including the use of AI-driven coaching, is isolated so that any problem encountered by the external application would not affect the ability to record trades and access core analytics.

Scalability

This architecture enables the user-facing application, logic processing, analysis and storage components to evolve separately as the usage grows.

The boundaries at the higher level of components also enable individual functions to be split up into separate services when needed due to performance concerns.

Hardware Constraints

TradeIO is not dependent on any sensors, actuators, or any other specialized hardware.

The user interface of the program is based on the use of a regular computer connected to the network.

Thus, the client-server architecture helps to avoid hardware dependence.

Rejected Pattern: Microservices

The microservices architecture was thought about because it would allow for independent deployment of services for trading, analysis, notifications, authentication, and other capabilities.

However, using this architecture would make the development more complicated from a deployment, networking, monitoring, testing, and consistency standpoint.

Given that this Senior Design team only consists of two people, this extra complexity cannot be warranted at this point in time.

The chosen architecture provides enough structure while allowing for easy migration to the service-based one.

Rejected Pattern: Embedded Sensor-Actuator

An embedded sensor-actuator design was also examined but does not fit the requirements of TradeIO.

TradeIO mainly deals with software-based input and stored information from the user and not from physical sensors or actuators.

Hence, there is no point including such constraints that will not be helpful for the project's objectives.

7. Decision Log

Decision	Alternatives Considered	Reason for Selection
Separate core analytics from optional AI insights.	Use AI for all analysis; use only deterministic analytics; combine deterministic analytics with optional AI coaching.	Core metrics and risk checks must remain available even if an external AI service is unavailable. AI is therefore reserved for higher-level explanatory and coaching functions.
Use a client-server architecture.	Peer-to-peer design; local-only application; client-server architecture.	TradeIO requires centralized persistent records, authentication, and access across sessions. Client-server provides a clear separation between the user-facing application and server-side processing.
Use an external identity provider.	Build credential and session management within TradeIO; use an external identity provider.	Authentication is security-sensitive but is not the primary innovation of TradeIO. Keeping identity management external allows the team to focus development effort on trading behavior, accountability, risk, and analysis.
Use centralized persistent application data.	Store data primarily in the browser; use separate storage for each feature; use a centralized persistent data store.	Trades, journals, settings, goals, and analysis information need consistent access across sessions. Centralized persistence provides the application with a common source of truth.
Separate notification delivery from core trade processing.	Use only in-app alerts; directly embed external notification delivery into trade processing; use a separate notification service.	External delivery can fail independently and may change in the future. Separating it prevents notification problems from blocking or corrupting core trade records.